

Bringing Private Reads to Hyperledger Fabric via Private Information Retrieval

ARTUR IASENOVETS ¹, FEI TANG ¹, HUIHUI ZHU ², PING WANG ² and LEI LIU ²

¹School of Cybersecurity and Information Law, Chongqing University of Posts and Telecommunications, Chongqing 400065, China

²School of Computer Science and Technology, Chongqing University of Posts and Telecommunications, Chongqing 400065, China

CORRESPONDING AUTHOR: FEI TANG (email: tangfei@cqupt.edu.cn).

This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFF0704102, in part by the Key Project of Science and Technology Research by Chongqing Education Commission under Grant KJZD-K202400610, and in part by the Chongqing Natural Science Foundation General Project under Grant CSTB2025NSCQ-GPX1263.

ABSTRACT Permissioned blockchains ensure integrity and auditability of shared data but expose query parameters to peers during read operations, creating privacy risks for organizations querying sensitive records. This paper proposes a Private Information Retrieval (PIR) mechanism to enable *private reads* from Hyperledger Fabric’s world state, allowing endorsing peers to process encrypted queries without learning which record is accessed. We implement and benchmark a PIR-enabled chaincode that performs ciphertext–plaintext (*ct×pt*) homomorphic multiplication directly within *evaluate* transactions, preserving Fabric’s endorsement and audit semantics. The prototype achieves an average end-to-end latency of 113 ms and a peer-side execution time below 42 ms, with approximately 2 MB of peer network traffic per private read in development mode—reducible by half under in-process deployment. Storage profiling across three channel configurations shows near-linear growth: block size increases from 77 kilobytes to 294 kilobytes and world-state from 112 kilobytes to 332 kilobytes as the ring dimension scales from 8,192 to 32,768 coefficients. Parameter analysis further indicates that ring size and record length jointly constrain packing capacity, supporting up to 512 records of 64 bytes each under the largest configuration. These results confirm the practicality of PIR-based *private reads* in Fabric for smaller, sensitive datasets and highlight future directions to optimize performance and scalability.

INDEX TERMS Private Information Retrieval (PIR), Hyperledger Fabric, Private Reads, Query Privacy.

I. INTRODUCTION

A. MOTIVATION AND PROBLEM STATEMENT

Blockchain [1] is a peer-to-peer network protocol that uses cryptographic primitives and a consensus mechanism to create and maintain a distributed ledger of transactions, such as those in Ethereum [2] and Hyperledger Fabric [3].

Hyperledger Fabric (HLF or Fabric) is widely adopted in various domains, one of them being Cyber Threat Intelligence (CTI) sharing [4]–[7], where multiple organizations exchange threat indicators and wish to keep their queries confidential not to be associated with specific threats or incidents. Blockchain guarantees, however, primarily cover data integrity and auditability, not query privacy. This issue was recently highlighted by Ethereum’s privacy special interest group in their “PSE Roadmap: 2025 and Beyond” [8], where they also emphasized the need for *private reads* next to private writes and private voting.

In Hyperledger Fabric [3], the separation of *evaluate* and *submit* makes the read-privacy gap explicit: an *evaluate* call is a read-only proposal sent to endorsing peers, which execute the chaincode and return results without committing to the ledger [9]. Crucially, these peers still observe all function arguments and read-sets, creating a privacy risk if queries are sensitive.

This motivates us to target the *read* privacy issue in Fabric, defined as the ability to hide which record is being queried from endorsing peers during *evaluate*-type calls. Given the problem description, an obvious approach is to construct and use a Private Information Retrieval (PIR) [10] protocol, which would enable a client to retrieve an item from a world state database without revealing to the peer which item was requested. However, integrating PIR into Fabric’s architecture presents unique challenges, as discussed next.

B. PIR SCHEME SELECTION

Private Information Retrieval (PIR) spans *information-theoretic* (IT-PIR) and *computational* (CPIR) designs, and each can be either *non-preprocessing* or *preprocessing* [10]. IT-PIR achieves unconditional privacy with multiple non-colluding servers [11]–[14], which is hard to guarantee in consortium ledgers. Single-server CPIR without preprocessing (e.g., SealPIR, FastPIR, OnionPIR, Spiral [15]–[18]) uses homomorphic encryption and fits Fabric’s one-round *evaluate* path, avoiding per-client state but incurring higher computational costs. Preprocessing CPIR (SimplePIR, Piano, HintlessPIR, YPIR; and other doubly-efficient variants [19]–[26]) reduces online cost via client/server “hints,” yet in Fabric this implies large client downloads or per-client peer state, conflicting with stateless, deterministic transactions. Consequently, we adopt the standard design pattern of single-server, HE-based, non-preprocessing CPIR construction for *private reads*, while *not claiming to instantiate* any existing PIR scheme verbatim. This preserves Fabric’s endorsement semantics, avoids hint management, and aligns with a single query–response interaction. While compute- and packing-limited, this choice is practical for smaller, sensitive datasets in permissioned settings, as our evaluation shows in V.

C. RELATED WORK

We now review prior efforts to integrate PIR with blockchain and distributed systems to enhance query privacy. A detailed comparison is provided in Table 6 in Section VI.

Xiao et al. [27] propose Cloak, a privacy-preserving blockchain query scheme that uses distributed point functions (DPF) and noise-based sub-requests to hide the retrieval information, achieving communication costs of 0.2–0.5 KB and query latency of 0.4–0.5 ms for databases with 4–64 records of 16–32 bytes each.

Mazmudar et al. [28] propose Peer2PIR, a privacy solution for IPFS using a combination of PAILLIERPIR and RL-WEPIR schemes to hide query content across peer routing, provider advertisements, and content retrieval functionalities. For the relevant private content retrieval function, they achieve communication costs of 10–15 MB and latencies of 0.5–15 s for databases of 1,000 to 1,000,000 records with 256 KB content blocks using the Spiral [18] protocol.

Kaihua et al. [29] design an SPV protocol for lightweight Bitcoin clients using a hybrid PIR scheme (combining IT-PIR and C-PIR) to enhance query privacy over Bloom-filter-based methods. For a single transaction verification in the weekly database, it achieves a communication cost of 666 KB and latency of 2.84 s for databases with 7,688–512,460 records of 62–876 bytes each.

Kumar et al. introduced a series of HLF frameworks that integrate Private Information Retrieval (PIR) across different domains, namely BRON [30] and DEBPIR [31]. Although posed as enabling PIR functionality, their evaluations primarily focus on the system’s write throughput rather than the specific performance of PIR queries.

Targeted gap. Permissioned ledgers like Hyperledger Fabric ensure integrity and auditability of shared data but leak function arguments and read-sets to endorsing peers during *evaluate* calls, creating privacy risks in multi-organisation settings. While prior works have explored PIR integration in various blockchain and distributed systems contexts, none have specifically addressed the challenge of enabling *private reads* in Hyperledger Fabric without modifying its core architecture. To fill this gap, we explore how to enable *private reads* from Fabric’s world state by integrating a HE-based PIR mechanism directly into chaincode, and we evaluate its practicality through detailed benchmarks and parameter studies.

D. KEY OBJECTIVE AND CONTRIBUTIONS

In this work, we demonstrate that HE-based PIR can be implemented natively in chaincode to enable *private reads* in Hyperledger Fabric under *evaluate* calls and achieve practical performance under certain parametrization choices.

Hence, the main contributions of this work are:

- 1) **Enabling Private Reads in Hyperledger Fabric.** We introduce the first end-to-end design, implementation and evaluation of a private information retrieval (PIR) mechanism, leveraging homomorphic encryption, natively integrated into Hyperledger Fabric chaincode. This enables clients to perform *private read* operations on the world state through *evaluate* transactions, ensuring the queried key remains confidential from endorsing peers. Our approach requires no modifications to Fabric’s core, demonstrating a practical, *plug-in privacy layer* for state queries.
- 2) **Polynomial database construction.** We formalize the mapping between Fabric’s key–value world state and a homomorphically encodable polynomial database, deriving feasibility constraints that link record length, ring dimension, and slot allocation. These relationships ensure the correctness of homomorphic retrieval and guide practical parameter selection for real-world datasets.
- 3) **Multi-channel approach.** To overcome database size limits imposed by polynomial packing constraints, we propose a multi-channel architecture where each channel hosts a specific HE parameter set and corresponding polynomial database instance, enabling clients to select appropriate channels based on their query needs.
- 4) **Comprehensive evaluation.** We implement a prototype using the Lattigo [32] library and benchmark the cryptographic operations, blockchain interactions, and end-to-end query latency under various configurations. Our results show that single-query latencies remain practical for typical Fabric deployments.
- 5) **Open-Source Release.** To encourage reproducibility, we release the full implementation of our Fabric chaincode, client logic and benchmarks at the repository: https://github.com/iasenovets/2_2_HLF_CPIR.

E. ORGANIZATION

The remainder of this paper is organized as follows: Section II reviews Fabric’s privacy features, database limits, homomorphic encryption-based PIR, and notation. Section III describes our system and threat models, polynomial database construction, feasibility constraints, multi-channel architecture, workflow and algorithms. Section IV analyzes security. Section V shows experimental setup, cryptographic and blockchain benchmarks, and overall system performance. Section VI discusses limitations and future directions, and Section VII concludes the paper.

II. PRELIMINARIES

A. FABRIC NATIVE PRIVACY FEATURES

We hereby acknowledge several native solutions for privacy that Hyperledger Fabric provides and explain why they do not fully address the query privacy problem.

Separate Channels. Fabric’s multi-channel architecture [9] isolates ledgers across subgroups of organizations, limiting which participants observe which data. However, channel separation controls *who* sees a ledger, query intent remains visible to all endorsers of a channel.

Private Data Collections (PDC). PDCs [9] restrict which organizations store and access private key–value pairs. The shared ledger records only hashes, while members of the collection hold plaintext. PDCs provide access control but still expose function arguments to endorsers inside the collection, leaving query patterns observable.

Fabric Private Chaincode (FPC). FPC [9], [33] executes chaincode within Intel SGX enclaves. Arguments and state are protected even from peer operators, but this requires Trusted Execution Environments (TEEs) and attestation, introducing additional hardware and trust assumptions.

B. FABRIC STORAGE LIMITS

We here address the question of how large a database can be stored in Fabric world state and ledger history, as this impacts the practical limits of our PIR construction.

World state (LevelDB/CouchDB). Fabric imposes no hard limit on the number of key–value entries in world state; capacity depends on available disk space and peer I/O throughput. In our implementation, each channel maintains a few small artifacts in world state that amount up to a 350 KB under typical parameters (see Section V), LevelDB is sufficient and preferable for performance and simplicity, while CouchDB [9] remains an option to extend from 8 MB to 4 GB if needed.

Ledger history (blockchain log). According to the documentation [9], block size is constrained by the ordering service configuration. By default, the Fabric orderer limits the serialized payload to *AbsoluteMaxBytes* = 10 MB (recommended under 49 MB given the gRPC ceiling of 100 MB), and typically aggregates up to *MaxMessageCount* = 500 transactions per block or *PreferredMaxBytes* = 2 MB. In our system, these limits affect only *submit* transactions such

as *InitLedger* or record updates. *Evaluate* transactions (including *PIRQuery*) are read-only and do not generate blocks, thus unaffected by ordering or batching constraints.

Implication. The effective capacity of a channel is governed primarily by cryptographic feasibility defined in Section III, and the size of a single world-state value (i.e., m_{DB}), rather than by Fabric’s block or database limits. For very large objects, an optional extension is to store them off-chain in IPFS [34] while persisting only their content identifiers (CIDs) in world state, keeping the polynomial m_{DB} as the structured component used for private retrieval.

C. HE-BASED PIR

Homomorphic Encryption. Lattice-based homomorphic encryption schemes, such as Brakerski/ Fan-Vercauteren (BFV), Brakerski-Gentry-Vaikuntanathan (BGV), CKKS and TFHE [35]–[38] have emerged as foundational technologies for practical CPIR implementations.

We base our CPIR construction on the BGV scheme, a lattice-based homomorphic encryption based on the Ring Learning With Errors (RLWE) problem [39], which supports both addition and multiplication over ciphertexts. The BGV [36] scheme defines operations over two polynomial rings: a ciphertext ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$ and a plaintext ring $R_T = \mathbb{Z}_T[X]/(X^N + 1)$, both sharing the same dimension $N = 2^{\log N}$. In our implementation, these rings are jointly specified by a single parameter literal $(\log N, \log Q_i, \log P_i, T)$ as provided by the Lattigo library [32] and paper [40]. The field T determines R_T , while the modulus chain (Q, P) and their bit-lengths $(\log Q_i, \log P_i)$ determine R_Q .

CPIR Definition. Model the database as a vector $D = \{d_0, \dots, d_{n-1}\}$. To retrieve desired record d_i without revealing i , the client forms a one-hot selection vector $\hat{v}_i$ defined as $\hat{v}_i = (0, \dots, 0, 1, 0, \dots, 0)$ where 1 corresponds to the desired record at index i . He then encrypts it as $ct_q = \text{Enc}_{pk}(\hat{v}_i)$ and sends ct_q to the server, which computes $ct_r = \text{Eval}(ct_q, D) = \text{Enc}(d_i)$, returning ct_r to the client for decryption $d_i = \text{Dec}_{sk}(ct_r)$ using his secret key sk , thus obtaining the desired record d_i without revealing i to the server, or peer in our case.

CPIR Instantiation: We instantiate Computational PIR as a tuple of algorithms $(\text{KeyGen}, \text{Enc}, \text{Eval}, \text{Dec})$:

- $\text{KeyGen}(\lambda) \rightarrow (pk, sk)$: On input the security parameter λ , output a public key pk and a secret key sk .
- $\text{Enc}_{pk}(\hat{v}_i) \rightarrow ct_q$: Given a record-level one-hot selector $\hat{v}_i$, expand it into $v_i \in \{0, 1\}^N$, then encode it into the plaintext ring R_T , and encrypt it as ct_q under pk .
- $\text{Eval}(ct_q, m_{DB}) \rightarrow ct_r$: Given ct_q and the plaintext polynomial database $m_{DB} \in R_T$, homomorphically evaluate the product to obtain an encrypted response $ct_r \in R_Q$.
- $\text{Dec}_{sk}(ct_r) \rightarrow d_i$: Using secret key sk , decrypt and decode ct_r , then extract window J_i to recover the desired record d_i .

Correctness. Given a database $D = \{d_0, \dots, d_{n-1}\}$ stored as a plaintext polynomial m_{DB} in R_T , for any index $i \in [n]$ and its corresponding one-hot selector $\hat{v}_i$, with Enc and Dec understood as above, the PIR scheme satisfies correctness if:

$$Dec_{sk}(Eval(Enc_{pk}(\hat{v}_i), m_{DB})) = d_i.$$

Remark (restricted operation set). The full BGV scheme also provides $EvalKeyGen$ to generate relinearization and rotation keys, supporting ciphertext–ciphertext multiplication ($ct \times ct$), automorphisms, and modulus switching. Since the database m_{DB} is stored in plaintext within world state, our PIR construction only requires homomorphic ciphertext–plaintext multiplication ($ct \times pt$). Further extension to ciphertext–ciphertext operations is possible but incurs additional overhead, as discussed in Section VI.

D. NOTATION

We summarize the main notation used throughout the paper in Table 1.

III. PROPOSED SYSTEM

A. SYSTEM MODEL

We introduce a blockchain-based query privacy system designed for permissioned ledgers. The system enables clients to perform *private reads* from the ledger while endorsing peers can evaluate read-only queries over encrypted inputs without learning which record was accessed. We achieve this by integrating a lattice-based CPIR scheme based on the BGV [36] homomorphic encryption scheme directly into Fabric chaincode. This approach ensures that clients remain the sole holders of decryption keys, while peers perform only black-box computations, thereby enhancing overall privacy without requiring trusted hardware or protocol modifications. Our system is composed of the following entities:

- **Data Owner (DO):** Endorsing peers that hold the current plaintext polynomial m_{DB} in world state and execute PIR during *evaluate*. DO is honest-but-curious.
- **Data Writer (DW):** A client organization that provisions or refreshes the database. DW invokes *submit* to initialize the ledger (e.g., set n and template bounds). Chaincode computes $record_s$, packs $D = \{d_0, \dots, d_{n-1}\}$, encodes it into m_{DB} , and persists it.
- **Data Requester (DR):** A client that privately retrieves a record. DR runs $KeyGen(\lambda) \rightarrow (pk, sk)$, forms $ct_q = Enc_{pk}(v_i)$, calls *evaluate* PIRQuery, and later decrypts ct_r .
- **Gateway (GW):** The Fabric client/chaincode interface used by DW and DR to invoke *InitLedger*, *GetMetadata*, and *PIRQuery*. It follows standard Fabric semantics; no extra trust is assumed.

Remark (world-state scope). In Fabric, the “ledger” comprises the blockchain log and the world state. Our CPIR operates on the world state: m_{DB} encodes the latest key-value snapshot, not the historical transaction logs. Both

TABLE 1. Notation

Symbol	Description
λ	Security parameter
pp	Public parameters (metadata)
n	Database size; index domain $[n] = \{0, \dots, n-1\}$
D	Database records $\{d_0, \dots, d_{n-1}\}$
m_{DB}	Encoded plaintext polynomial representation of D
$\hat{v}_i$	Record-level one-hot selector for index i
v_i	Windowed logical-slot selector for index i with $record_s$ contiguous ones
c	Logical-slot vector $(c_0, \dots, c_{N-1})$
J_i	Disjoint logical-slot window for index i
$Enc_{pk}(\cdot), Dec_{sk}(\cdot)$	Encrypt / Decrypt
$Eval(\cdot)$	Homomorphic evaluation (ct-pt multiply)
pk, sk	Public / secret keys
ct_q	Encrypted query $Enc_{pk}(\hat{v}_i)$
ct_r	Encrypted response $Eval(ct_q, m_{DB})$
d_i	Decrypted record $Dec_{sk}(ct_r)$
$KeyGen(\lambda)$	Key generation $\rightarrow (pk, sk)$
N	Ring dimension
$\log N$	Logarithm base 2 of ring dimension N
$\log Q_i$	Bit-lengths of primes forming modulus chain Q
$\log P_i$	Bit-lengths of special primes P
T	Plaintext modulus
$record_s$	Slots allocated per record
$record_b$	Base serialized size of a record in bytes
$record_{\mu, \log N}$	Template-specific minimum
$\mathcal{S}$	Allowed discrete slot sizes
$ \cdot , size(\cdot)$	Length in elements / bytes
$\mathcal{C}(N, s, n)$	Capacity predicate: $n \cdot s \leq N$
$\mathcal{M}(\log N, s)$	Template predicate: $s \geq record_{\mu, \log N}$
$\mathcal{D}(s)$	Discrete predicate: $s \in \mathcal{S}$
$\mathcal{F}(\log N, s, n)$	Feasibility tuple: $\mathcal{C} \wedge \mathcal{M} \wedge \mathcal{D}$
DO, DW, DR, GW	Data Owner; Data Writer; Data Requester; Gateway
<i>evaluate, submit</i>	Fabric read / write transaction types
$\mathcal{L}$	Leakage considered (ciphertext size, protocol timing)

world state and *ledger* capacity concerns are addressed in Section II.

B. SECURITY ASSUMPTIONS AND THREAT MODEL

Our design follows the standard *honest-but-curious* adversarial model. We explicitly consider the following assumptions and threats:

- **Endorsing peers ($\mathcal{DO}$).** Execute chaincode correctly but may try to infer the queried index from *evaluate* inputs or logs. They see ct_q , metadata (pp), and m_{DB} .
- **Data Writer ($\mathcal{DW}$).** Issues initialization writes via *submit*. $\mathcal{DW}$ is not trusted with decryption keys and learns nothing about $\mathcal{DR}$'s queries. We assume $\mathcal{DW}$ follows the write protocol but is not relied upon for privacy.
- **External observers.** May eavesdrop on client-peer traffic. Without sk , ct_q and ct_r reveal nothing under BGV assumptions.

Security Objective (Private Reads). For any $i \in [n]$, neither $\mathcal{DO}$ nor external observers can distinguish which d_i is requested from ct_q and ct_r . The only permissible leakage is ciphertext size and protocol timing, denoted collectively as $\mathcal{L}$. We provide a security proof in Section IV.

C. SYSTEM OVERVIEW

The proposed system integrates computational Private Information Retrieval (CPIR) directly into Hyperledger Fabric chaincode. Its purpose is to ensure that query indexes remain hidden from endorsing peers while preserving Fabric's endorsement and audit workflow. At a high level, the workflow consists of four stages, illustrated in Fig. 1.

- 1) **Ledger initialization.** $\mathcal{DW}$ invokes `InitLedger` via $\mathcal{GW}$ using *submit*. Chaincode derives $record_s$ from $record_b$, packs D into $c = (c_0, \dots, c_{N-1})$, encodes m_{DB} , and stores m_{DB} and metadata in world state held by $\mathcal{DO}$.
- 2) **Metadata discovery.** $\mathcal{DR}$ calls `GetMetadata` via *evaluate* to obtain n , $record_s$, and BGV parameters needed to form a valid query.
- 3) **Private retrieval.** $\mathcal{DR}$ constructs $ct_q = \text{Enc}_{pk}(v_i)$ and invokes `PIRQuery` via *evaluate*. $\mathcal{DO}$ computes $ct_r = \text{Eval}(ct_q, m_{DB})$ and returns it.
- 4) **Decryption.** $\mathcal{DR}$ decrypts ct_r to recover $d_i = \text{Dec}_{sk}(ct_r)$.

D. POLYNOMIAL DATABASE CONSTRUCTION

To enable PIR queries over structured ledger data, we must embed records into a plaintext polynomial m_{DB} suitable for BGV evaluation. Our prototype adopts a fixed-width packing strategy, which proceeds in four steps.

Step 1: Serialize to Byte Array. Each structured record d_i is serialized into a byte array using a deterministic JSON-to-bytes scheme. Every character is represented by its ASCII code in $[0, 255]$, which is also our plaintext modulus T . Although we set $T = 65537$ as default in Section V, we use $T = 256$ as an example here for clarity.

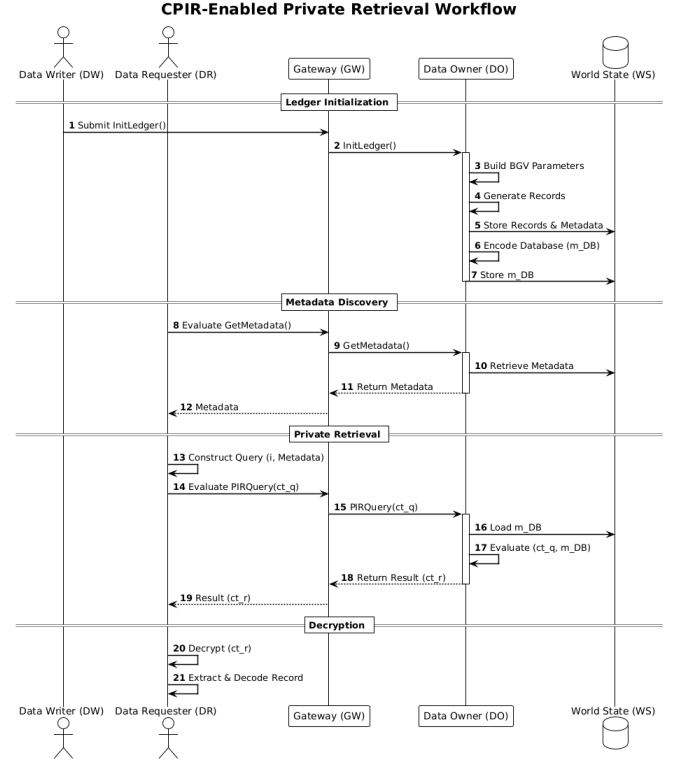


FIGURE 1. “Workflow. $\mathcal{DW}$ initializes the ledger via $\mathcal{GW}$, which triggers chaincode on endorsing peers ($\mathcal{DO}$). $\mathcal{DO}$ executes the protocol and persists state in world state (m_{DB} , metadata, JSON records). $\mathcal{DR}$ later obtains metadata, submits $ct_q = \text{Enc}_{pk}(v_i)$, $\mathcal{DO}$ evaluates $ct_r = \text{Eval}(ct_q, m_{DB})$ against world state, and $\mathcal{DR}$ decrypts to d_i .

Step 2: Calculate Slot Window. To achieve uniform packing, we determine a fixed slot window $record_s$ across all records, where $record_b$ is the maximum serialized record length in bytes and $bytesPerSlot$ is the number of bytes stored per slot. We first calculate the basic slot requirement and then apply a discrete rounding policy:

$$record_s = 8 \cdot \left\lceil \frac{record_b}{8 \cdot bytesPerSlot} \right\rceil, \quad (1)$$

where $bytesPerSlot = \frac{\log_2(T)}{8}$ given plaintext modulus T . With $T = 256$, $\log_2(256) = 8$ and thus $bytesPerSlot = 1$ byte. For example, if the largest record is 126 bytes, then the basic requirement is $\lceil 126/1 \rceil = 126$ slots, and after discrete rounding $record_s = 8 \cdot \lceil 126/8 \rceil = 8 \cdot 16 = 128$ slots.

Step 3: Pack into Logical Slot Vector. Each serialized record d_i is packed into its disjoint window $J_i = \{i \cdot record_s, \dots, (i+1) \cdot record_s - 1\}$ in the logical-slot vector $c = (c_0, c_1, \dots, c_{N-1})$:

$$c[J_i[k]] = c[i \cdot record_s + k] = \begin{cases} d_i[k] & \text{if } k < |d_i| \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

for $i \in [n]$ and $k \in [0, record_s - 1]$, where $d_i[k]$ denotes the k -th byte of record d_i , and $J_i[k] = i \cdot record_s + k$ is the k -th slot index in window J_i . Each $c_j \in [0, T - 1]$ is the j -th coordinate of the logical-slot vector c .

Padding zeros are added if a record is shorter than $record_s$. Thus each record d_i occupies a contiguous slot interval that can be privately retrieved through PIR.

Step 4: Encode into Plaintext Polynomial. Finally, BGV batching encoder maps vector c to a plaintext polynomial:

$$m_{DB}(X) = \sum_{j=0}^{N-1} \tilde{c}_j X^j \in R_T, \quad (3)$$

where $R_T = \mathbb{Z}_T[X]/(X^N + 1)$. Here, $\tilde{c}$ denotes the polynomial coefficients produced when BGV encodes the logical-slot vector c ; generally, $\tilde{c} \neq c$. Encoding the query with the same encoder makes *Eval* decode slotwise as $y_j = (v_i)_j c_j$. This polynomial serves as the database representation m_{DB} in the PIR protocol, and the client extracts the logical slots corresponding to the requested record.

E. FEASIBILITY CONSTRAINTS

Embedding records into the plaintext polynomial m_{DB} is feasible only for parameter triples $(\log N, n, record_s)$ that satisfy *all* of the following constraints.¹

Constraint 1: Ring capacity. The total number of occupied slots cannot exceed the ring size:

$$n \cdot record_s \leq N. \quad (4)$$

This represents the fundamental mathematical limit imposed by the cryptographic parameters. For example, with $\log N = 13$ ($N = 8192$) and $record_s = 224$, at most $\lfloor 8192/224 \rfloor = 36$ records can be packed.

Constraint 2: Template-specific minima. We anticipate that different application scenarios require different record templates composed of distinct field combinations. Each template μ imposes a minimum slot requirement $record_{\mu, \log N}$, determined by its mandatory fields:

$$record_{\mu, \log N} = B_\mu + \sum_{i \in \mathcal{H}_\mu} |F_i| + O_\mu, \quad (5)$$

where B_μ is the base structure size for template μ , $\mathcal{H}_\mu \subseteq F_1, F_2, \dots, F_L$, denotes the set of fields included in template μ , $|F_i|$ is the byte length of field F_i , O_μ captures serialization overhead and metadata specific to template μ .

Constraint 3: Discrete allocation policy. Operationally, we restrict the slot window $record_s$ to a discrete set for implementation simplicity:

$$record_s \in \mathcal{S}, \mathcal{S} = \{64, 128, 224, 256, 384, 512\} \text{ slots}. \quad (6)$$

This means that only certain fixed slot sizes are allowed, simplifying client query construction and chaincode logic.

Constraint hierarchy and feasibility. We summarize the three constraints as predicates:

$$\mathcal{C}(N, s, n) : n \cdot s \leq N, \quad (7a)$$

$$\mathcal{M}(\log N, s) : s \geq record_{\mu, \log N}, \quad (7b)$$

$$\mathcal{D}(s) : s \in \mathcal{S}. \quad (7c)$$

¹The complete slot-packing workflow and parameter feasibility plots are given in the Supplementary Material (Figs. S1–S2).

where $\mathcal{C}$ captures ring capacity, $\mathcal{M}$ captures template minima, and $\mathcal{D}$ captures discrete allocation, s denotes $record_s$ here for brevity. The overall feasibility condition is:

$$\mathcal{F} \iff \mathcal{C}(N, s, n) \wedge \mathcal{M}(\log N, s) \wedge \mathcal{D}(s) \quad (8)$$

F. MULTI-CHANNEL ARCHITECTURE

The packing strategy and feasibility constraints highlight an important observation: no single homomorphic parameter set can efficiently support the full diversity of dynamic CTI records encountered in practice. Compact records fit comfortably under smaller rings, while full JSON objects with long cryptographic hashes exceed the slot budget of these configurations. To balance scalability and expressiveness, we design a *multi-channel architecture* in Hyperledger Fabric, where each channel is provisioned with a distinct BGV parameter set and record template.

a) Channel Mini ($N = 2^{13}$). Supports compact records with maximum scalability and lowest query latency. For example, with $N = 8192$ slots, the system accommodates up to 128 records when $\max_i |d_i| \leq 64$ bytes, and 16 records when $\max_i |d_i| \leq 512$ bytes.

b) Channel Mid ($N = 2^{14}$). Targets medium-sized records that include MD5 and truncated SHA-256 fields alongside classification metadata. With $N = 16384$ slots, the system supports up to 256 records at ≤ 64 bytes or 32 records at ≤ 512 bytes.

c) Channel Rich ($N = 2^{15}$). Handles the most detailed records, including full-length hashes and multiple metadata fields. Here, $N = 32768$ slots allow up to 512 records at ≤ 64 bytes or 64 records at ≤ 512 bytes.

Channel semantics. As shown in Fig. 2, each channel maintains its own PIR chaincode instance and world state. The world state contains:

- The *polynomial view*: the packed plaintext polynomial m_{DB} under key “ m_{DB} ”.
- The *normal view*: JSON records stored optionally under keys “ $record\%03d$ ” for auditability.
- *Metadata*: “ n ”, “ $record_s$ ” and “ bgv_params ”.

G. WORKFLOW DETAILS

The proposed system has five main routines, corresponding to 3 chaincode-side functions (*InitLedger*, *GetMetadata*, *PIRQuery*) and 2 client-side utilities (*FormSelectionVector*, *DecryptResult*). The detailed pseudocode for these routines is provided in the Supplementary Material (Algorithms S1–S5). Figure 1 provides the high-level overview. A Data Writer (*DW*) provisions the database $D = \{d_0, \dots, d_{n-1}\}$, a Data Owner (*DO*) maintains the polynomial m_{DB} in world state and executes PIR evaluations, and a Data Requester (*DR*) retrieves d_i privately using homomorphic encryption. The detailed steps are as follows:

- 1) **DW submits initialization.** *DW* calls *InitLedger* (Alg. S1) via *GW* (*submit*) with inputs $(n, record_s^{DW})$

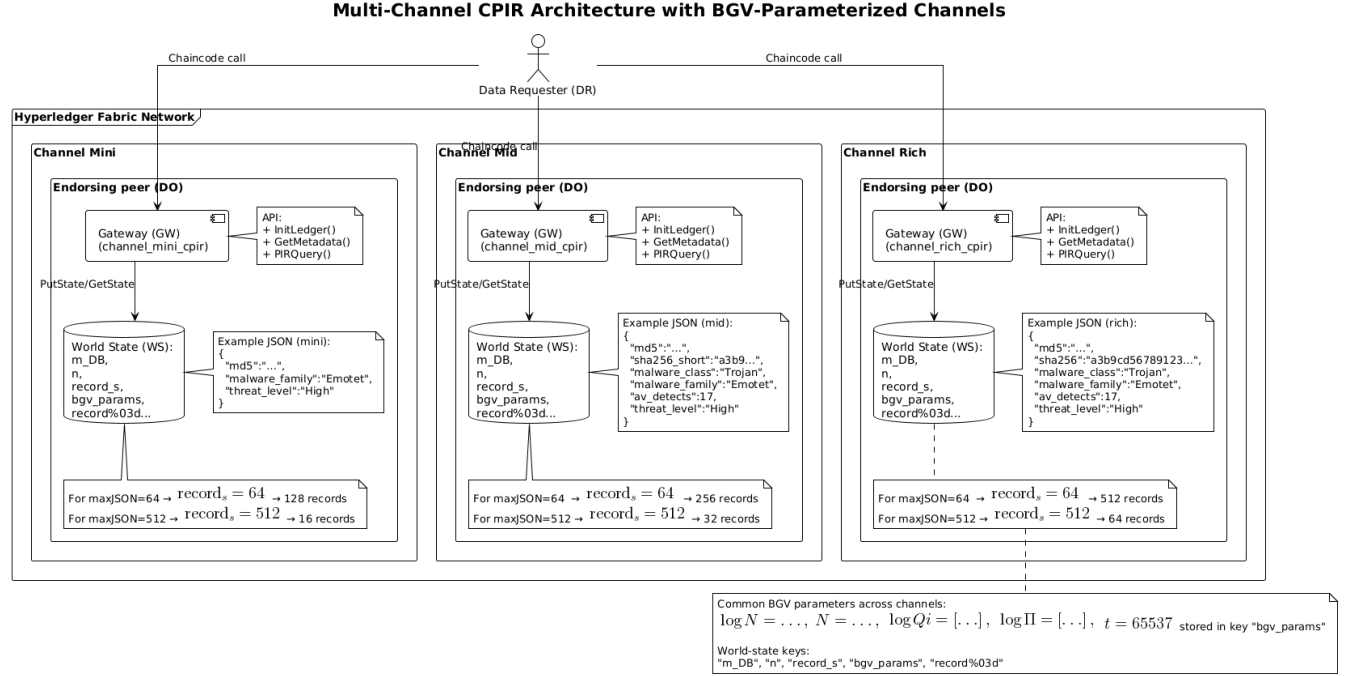


FIGURE 2. Multi-channel CPIR architecture. Each channel instantiates a separate CPIR chaincode and maintains its own m_{DB} polynomial, parameterized by $\log N$. This allows compact, mid-size, and rich CTI records to coexist under the same Fabric network.

- and an optional hint $(\log N, \log Q_i, \log P_i, T)$. Here $record_s^{DW}$ denotes the maximum JSON size anticipated by the writer.
- DO validates and derives parameters.** DO rounds the writer's input to a discrete slot size $record_s^{GW} = 8 \cdot \lceil record_s^{DW} / 8 \rceil$. If $\log N$ is absent, the smallest feasible $\log N$ is chosen such that $\mathcal{C}(N, record_s^{GW}, n)$ holds. BGV parameters $\{\log N, N, \log Q_i, \log P_i, T\}$ are constructed and stored.
 - DO prepares records.** Records $D = \{d_0, \dots, d_{n-1}\}$ are ingested or synthesized with $|d_i| \leq record_s^{DW}$. The definitive slot allocation is then fixed as $record_s = 8 \cdot \lceil \max_i |d_i| / 8 \rceil$ (discrete policy), checked against feasibility predicates $\mathcal{M}(\log N, record_s)$, $\mathcal{D}(record_s)$, $\mathcal{C}(N, record_s, n)$.
 - Pack and persist.** Each d_i is placed in a disjoint window $J_i = \{i \cdot record_s, \dots, (i+1)record_s - 1\}$ of $c = (c_0, \dots, c_{N-1})$, zeros pad unused slots, and c is encoded into the plaintext polynomial $m_{DB}(X)$ in (3) at the maximum level. World state stores "m_DB", "n", "record_s", and "bgv_params" plus optional "record%03d" entries.
 - DR discovers metadata.** DR calls GetMetadata (Alg. S2) via evaluate to obtain $(n, record_s, \log N, N, T, \log Q_i, \log P_i)$. This enables reconstruction of the cryptographic context.
 - DR instantiates crypto context.** From metadata, DR builds parameters, executes $KeyGen(\lambda) \rightarrow (pk, sk)$, and prepares encoder/encryptor objects.

- Form and encrypt query.** For index $i \in [n]$, DR runs FormSelectionVector (Alg. S3): define J_i , set v_i with ones on J_i , encode it as $m_q \in R_T$, encrypt as $ct_q = Enc_{pk}(m_q)$, and serialize/Base64-encode.
- PIR query evaluation.** DR issues PIRQuery(ct_q^{B64}) (Alg. S4) via evaluate. DO decodes, reloads m_{DB} if necessary, and computes $ct_r = Eval(ct_q, m_{DB})$, returning the Base64-encoded ciphertext.
- Decryption and reconstruction.** DR runs DecryptResult (Alg. S5) to recover response plaintext $m'(X)$, decode it into c , extract bytes from J_i , stop at padding zero, and reconstruct d_i .

Remark (Single-record PIR). The proposed system currently supports windowed selection vectors for single-record PIR queries. Extending to multi-record queries is feasible by modifying the selection vector to include multiple J_i .

IV. SECURITY ANALYSIS

We now demonstrate how our construction satisfies the security objective defined in Section III-B by reduction to the IND-CPA [41] security of the BGV scheme [36] (implemented via Lattigo [32]) under the RLWE assumption [39]. Our system implements a single-server, single-round HE-based PIR construction by tailoring Lattigo's PIR example [42] into a non-threshold, windowed-slot variant designed for the Hyperledger Fabric chaincode environment. This construction relies on the security guarantees of BGV and RLWE, which we leverage in our proof below.

Proof. Under the BGV scheme [36], the encrypted query $ct_q = \text{Enc}_{pk}(m_q)$ is a noisy ciphertext of the form $ct_q = (c_0, c_1) = (pk_0 \cdot u + T \cdot e_0 + m_q, pk_1 \cdot u + T \cdot e_1) \in R_Q \times R_Q$ that stores the encoded selection mask $m_q \in R_T$ for logical-slot selector $v_i \in \{0, 1\}^N$. Here, $u \leftarrow \mathcal{X}_s$, $e_{pk}, e_0, e_1 \leftarrow \mathcal{X}_e$ are small polynomials drawn from the secret and error distributions, respectively, and $pk = (pk_0, pk_1) = (a \cdot sk + T \cdot e_{pk}, -a)$ is the public key, where $a \leftarrow R_Q$ is uniformly random and $sk \leftarrow \mathcal{X}_s$ is the secret key. Since m_q is hidden by the fresh terms $(pk_0 \cdot u + T \cdot e_0)$ and $(pk_1 \cdot u + T \cdot e_1)$, under RLWE we have $ct_q \approx_c U$ for any adversary $\mathcal{A}$ running in time $\text{poly}(\lambda)$ without the secret key sk , where $U \leftarrow R_Q \times R_Q$ is a uniformly random element. Thus, ct_q reveals no information about m_q or its selected logical-slot window J_i .

Indistinguishability Game. Consider an adversary $\mathcal{A}$ running in time $\text{poly}(\lambda)$ and controlling an honest-but-curious endorsing peer $\mathcal{DO}$. The adversary $\mathcal{A}$ selects two indices $i, j \in [n]$, corresponding to mask polynomials $m_{q,i}$ and $m_{q,j}$ obtained by encoding v_i and v_j , whose logical slots are set to 1 on J_i or J_j and 0 elsewhere. A challenger $\mathcal{C}$ samples a random bit $b \leftarrow \{0, 1\}$ and sets $m_{q,b} = m_{q,i}$ if $b = 0$, or $m_{q,b} = m_{q,j}$ if $b = 1$. $\mathcal{C}$ then encrypts the query $ct_q \leftarrow \text{Enc}_{pk}(m_{q,b})$ and evaluates the protocol to obtain $ct_r = \text{Eval}(ct_q, m_{DB})$. After this, $\mathcal{C}$ provides the view $\text{View}_{\mathcal{A}} = \{ct_q, ct_r, m_{DB}, pp, \mathcal{L}\}$ to $\mathcal{A}$. Finally, $\mathcal{A}$ outputs a bit b' and wins if $b' = b$. Since ct_q is obtained through IND-CPA secure $\text{Enc}_{pk}(\cdot)$ and ct_r is the result of a deterministic $\text{Eval}(\cdot)$ using ct_q , pp and m_{DB} , the distribution of $\text{View}_{\mathcal{A}}$ depends on b only through the underlying $m_{q,b}$. Hence, any $\mathcal{A}$ running in time $\text{poly}(\lambda)$ capable of distinguishing $\text{View}_{\mathcal{A}}^{(i)}$ from $\text{View}_{\mathcal{A}}^{(j)}$ with non-negligible advantage can be reduced to an $\mathcal{A}'$ that breaks the IND-CPA security of the BGV scheme with the same advantage. This contradicts the RLWE assumption [39]. Thus, we conclude² query indistinguishability under the $(N, Q, \mathcal{X}_{s,e}, T)$ -RLWE assumption for any $\mathcal{A}$ running in time $\text{poly}(\lambda)$ to be: $\Pr[b' = b] \leq \frac{1}{2} + \text{negl}(\lambda)$.

Corollary 1 (Single Read Privacy). For a single query with uniformly sampled $i \in [n]$, an adversary $\mathcal{A}$ cannot recover i or determine which d_i is requested, since the view reveals no information about the position of the bit (J_i) in v_i beyond what is permitted in $\mathcal{L}$, and hence its guessing probability satisfies $\Pr[\mathcal{A}(\text{View}_{\mathcal{A}}) \in \{(i, d_i)\}] \leq \frac{1}{n} + \text{negl}(\lambda)$.

Corollary 2 (Repeated Reads). (1) Following the proof above, any $\mathcal{A}$ views each $ct_{q,i}^{(t)} \approx_c ct_{q,j}^{(t')}$, for any $i, j \in [n]$ and any round $t, t' \in \mathbb{N}$ (even if $i = j$ and/or $t \neq t'$). By a standard hybrid argument, any $\mathcal{A}$ running in time $\text{poly}(\lambda)$ sees $\text{View}_{\mathcal{A}}(\mathcal{Q}) \approx_c \text{View}_{\mathcal{A}}(\mathcal{Q}')$ for any two query sequences $\mathcal{Q}, \mathcal{Q}'$ of equal length. (2) Each evaluation $ct_r^{(t)} = \text{Eval}(ct_q^{(t)}, m_{DB})$ involves a ciphertext–plaintext multiplication of $ct_q^{(t)} = \text{Enc}_{pk}(m_q^{(t)})$ with m_{DB} over R_Q . This operation is independent of the mask $m_q^{(t)}$ and its corresponding $J_i^{(t)}$; by (1), the resulting view reveals nothing about i beyond $\mathcal{L}$ to any $\mathcal{A}$ running in time $\text{poly}(\lambda)$.

²See App. 6 in [43] and Cor. 2 in [20] for similar formal proofs.

V. PERFORMANCE EVALUATION

A. EXPERIMENTAL SETUP

All experiments were executed on a local Ubuntu 24.04 host running under WSL2 on an Intel Core i5-3380M CPU (2 cores/4 threads, 2.90 GHz) with 7.7 GB of RAM and a 1 TB SSD. During evaluation, the average available memory was 6.9 GB with a 2 GB swap partition, and the root filesystem reported 946 GB of free space. The software stack consisted of Go 1.24.1, Docker 27.4.0, and Docker Compose v2.31.0, hosting Hyperledger Fabric v2.5 with LevelDB as the world-state database and a Solo ordering service, using recommended settings (*BatchSize.AbsoluteMaxBytes*=99 MB, *PreferredMaxBytes*=2 MB per block). The network configuration comprised a single organization with one peer per channel, sufficient for privacy evaluation, though it can be extended to multiple peers and organizations as needed.

Our implementation employs Fabric GO SDK [44] for blockchain interactions and the Lattigo v6 library [32] as the homomorphic encryption backend. Lattigo provides a Go-native implementation of the BGV scheme [36], whose security and correctness have been validated in prior literature. Accordingly, our focus is on evaluating its *practical performance within a permissioned blockchain environment* for enabling *private reads* from world state, rather than re-verifying the theoretical properties of BGV itself.

Unless otherwise stated, each reported value represents the mean of 20 executions under both cold and warm cache conditions, cross-verified against peer logs for consistency.

B. CRYPTOGRAPHIC PERFORMANCE

Parameter Configuration. Table 2 summarizes the BGV parameter sets used in our evaluation, including the ring dimension N , ciphertext modulus chain $(\log Q_i, \log P_i)$, plaintext modulus T , slot allocation record_s , and database size n . Each configuration corresponds to one Fabric channel and maintains a feasible packing ratio as defined in Section III.

TABLE 2. Default BGV Parameter Configuration per Channel

N	$\log Q_i$	$\log P_i$	T	record_s	n
2^{13}	[54]	[54]	65537	128	64
2^{14}	[54]	[54]	65537	224	73
2^{15}	[54]	[54]	65537	256	128

Overall Results. Table 3 lists the measured execution time of core cryptographic operations (*KeyGen*, *Enc*, *Eval*, and *Dec*) for each evaluated ring dimension N .

TABLE 3. Execution Time of Cryptographic Operations (ms)

N	<i>KeyGen</i>	<i>Enc</i>	<i>Eval</i>	<i>Dec</i>
2^{13}	27.7	11.7	16.9	5.1
2^{14}	42.7	27.7	30.7	11.9
2^{15}	55.0	49.8	64.8	15.6

Figure 3 consolidates the main cryptographic evaluation metrics: (A) end-to-end latency by algorithmic stage, (B) serialized artifact size,³ and (C) slot utilization ratio within the packed database polynomial m_{DB} .

C. BLOCKCHAIN PERFORMANCE

Chaincode Execution Timings. Figure 4 visualizes the average server-side execution time of the main chaincode functions across three channels, each corresponding to a different ring size N .

Remark (execution paths). Transactions that modify world state (e.g., `InitLedger`) are issued via the *submit* path and committed through Solo ordering, while read-only operations (`GetMetadata`, `PIRQuery`) use the *evaluate* path, bypassing block creation and ordering.

Blockchain Performance. On-chain network behavior is reported in Table 4, which presents the average peer-side network I/O per `PIRQuery` transaction across the three channel configurations⁴.

TABLE 4. Peer network I/O per `PIRQuery` (avg, KB/tx)

Channel	N	NET I/O (KB/tx)
mini	2^{13}	524.62
mid	2^{14}	1042.09
rich	2^{15}	2072.88

D. OVERALL SYSTEM PERFORMANCE

End-to-End Workflow Summary. Based on the data in Table 3 and Figure 4, we now to summarize two main operational workflows in the system, involving $\mathcal{DW}$ and $\mathcal{DR}$, while $\mathcal{DO}$ is implicitly involved as the executing peer during private queries.

The first workflow corresponds to $\mathcal{DW}$ initializing the ledger through `InitLedger`, which packs and encodes the plaintext database m_{DB} into world state.

The second workflow corresponds to $\mathcal{DR}$ performing a private query by sequentially executing `GetMetadata` $\rightarrow$ `KeyGen` $\rightarrow$ `Enc` $\rightarrow$ `PIRQuery` $\rightarrow$ `Dec`. The PIR evaluation itself is performed by $\mathcal{DO}$ (endorsing peer) using ciphertext-plaintext multiplication during the evaluate transaction.

Table 5 summarizes the cryptographic and blockchain timings for both workflows, averaged across all three channel configurations.

TABLE 5. End-to-End Performance Analysis (ms)

Workflow	Cryptographic Operations (ms)	Blockchain Operations (ms)	Total Time (ms)
$\mathcal{DW}$'s Upload	0.0	220.0	220.0
$\mathcal{DR}$'s Query	82.4	30.5	112.9

³Extended cryptographic object sizes and world-state footprints are reported in Table S1–S2 of the Supplementary Material.

⁴For visual blockchain-side evaluation of the system across three channels, please refer to Figure S3 in the Supplementary Material.

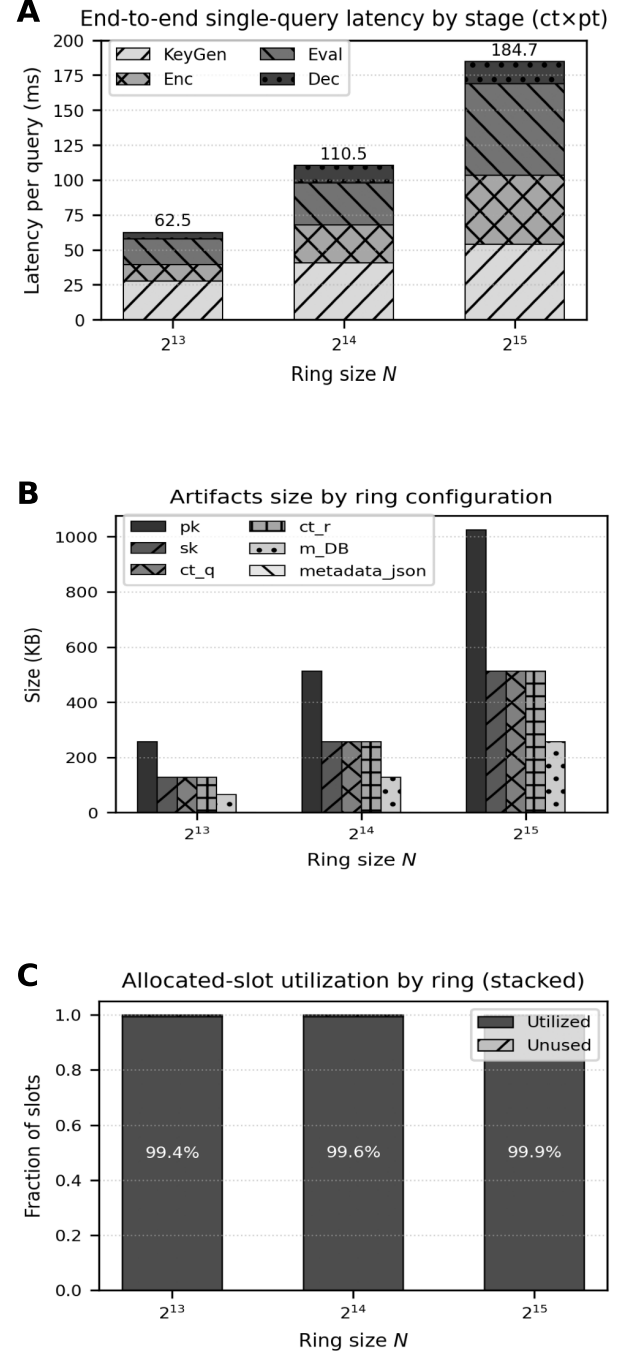


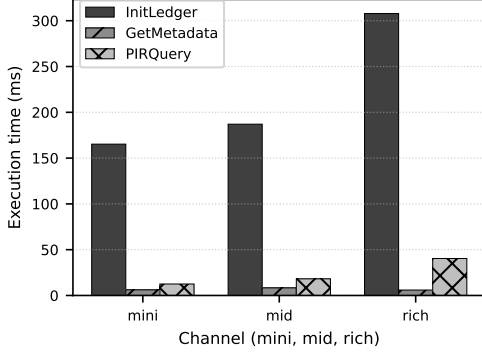
FIGURE 3. Cryptographic performance of the BGV-based CPIR system: (A) latency by algorithmic stage, (B) artifact size by ring configuration, and (C) allocated-slot utilization.

Projected Cost for PIR Query. Among our results, for the ring size 2^{15} configuration, issuing 100/1000/10000 private queries incurs $\approx 207\text{MB}$ / 2.07 GB / 20.7 GB of peer-side network I/O with an associated chaincode evaluation time of $\approx 0.07/0.67/6.73$ minutes, respectively, when using our protocol, as shown in Table 7 and Figure 5.

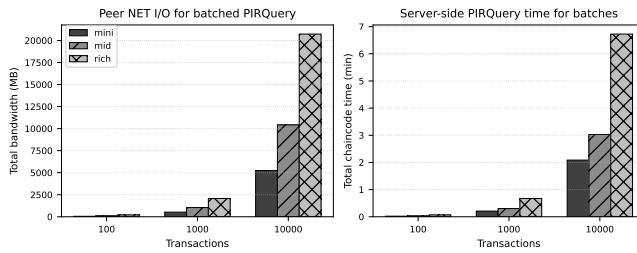
TABLE 6. Comparison with existing privacy-preserving query systems

Work	Targeted System	Privacy Technique	Execution Location	Targeted Operation	Comm. Cost / Query (MB)	E2E Query Time (ms)	Db Size / (# of records)	Record Size / length (bytes)
[31]	HLF	OT	on-chain	read / write	N/A	600	10,000	N/A
[30]	HLF	ZKP	on-chain	read / write	N/A	N/A	10,000	N/A
[27]	Blockchain	DPF	off-chain	read	0.0002-0.0005	0.4-0.5	4-64	16-32
[28]	IPFS	HE-PIR	off-chain	read	10-15	500-16,000	1,000-1M	256,000
[29]	Bitcoin	Hybrid PIR	off-chain	read	0.65	2840	7,688-512,460	62-876
Ours	HLF	HE-PIR	on-chain	read	1-2	112.9	16-512	64-512

Execution time of chaincode functions per channel (server-side avg)

**FIGURE 4.** Average chaincode execution time by function and ring size. Each bar represents the mean execution time over multiple epochs.**TABLE 7.** Projected cost for multiple PIR queries at 2^{15} (peer perspective)

# Transactions	Total Bandwidth (MB)	Total Time (min)
100	207.3	0.07
1000	2073.0	0.67
10000	20730.0	6.73

**FIGURE 5.** Projected cost for multiple PIR queries at 2^{15} (peer perspective)

Remark (on network I/O measurement). The network I/O values in Table 4 and Table 7 were obtained from *docker stats* snapshots collected for the *peer0.org1.example.com* and *orderer0.group1.orderer.example.com* containers during each benchmark epoch.

VI. DISCUSSION

A. FUTURE WORK AND OUTLOOK

Although our prototype validates this concept, several challenges remain in performance, scalability, and deployment. Future research should address (i) side-channel resilience through constant-time chaincode evaluation and standardized ciphertext serialization to mitigate timing and size-based leakages. (ii) Scaling beyond moderate database sizes will require some form of sublinear or sharded CPIR, similar to [20]–[23], yet adapted to Fabric’s architecture and resource constraints. Moreover, (iii) extending the current ciphertext–plaintext ($ct \times pt$) evaluation to fully encrypted-database ($ct \times ct$) computation would enable richer on-chain analytics under encryption, though it demands relinearization, rotation, and modulus-switching capabilities within Fabric’s resource limits. (iv) Dynamic ledger updates, (addRecord) operation would allow incremental growth without full reinitialization, raising questions around re-encoding and key management. (v) Selective field-level retrieval through adaptive packing could enable varying length records and partial access control. (vii) Deployment optimization is crucial. In our setup, the chaincode ran in *development mode* as an external service, causing duplicated gRPC transmissions for each ciphertext (ct_q , ct_r) due to two network hops—client↔peer and peer↔chaincode. This resulted in ≈ 2072 KB per query for $N = 2^{15}$, as shown in Table 7. Running the chaincode *in-process* within the peer would eliminate the second hop, halving the network cost to ≈ 1024 KB per query.

B. RELATED WORK COMPARISON

Table 6 contrasts our proposed PIR-based *private reads* mechanism with several recent works discussed in Section I, that integrate Private Information Retrieval (PIR) or related cryptographic mechanisms to achieve query privacy across different domains. The comparison focuses on six key aspects: (i) which target system the privacy solution is built for, (ii) the underlying privacy primitive (CPIR, IT-PIR, hybrid, DPF etc), (iii) whether privacy computation occurs on-chain (via smart contracts or chaincode) or off-chain, (iv) the targeted operation (read-only or read/write), (v) communication cost per query (in MB), (vi) end-to-end query time (in ms), (vii) database size (number of records), and (viii) record size (in bytes).

VII. CONCLUSION

This paper presented a Private Information Retrieval (PIR) mechanism for enabling *private reads* in Hyperledger Fabric, allowing endorsing peers to evaluate encrypted queries without learning which record was accessed. The proposed chaincode performs ciphertext–plaintext (*ct×pt*) homomorphic multiplication directly within *evaluate* transactions, preserving Fabric’s endorsement and audit semantics. Our prototype achieves an average end-to-end latency of 113 ms and a peer-side execution time below 42 ms, with approximately 2 MB of peer network traffic per query in development mode—reducible to about 1 MB under in-process deployment. Storage profiling shows near-linear growth in both block and world-state sizes as the ring dimension scales, and parameter analysis confirms practical support for up to 512 records of 64 bytes each under 2^{15} . These results validate the feasibility of PIR-based *private reads* in permissioned ledgers, offering millisecond-scale query latency and full compatibility with Fabric’s architecture.

REFERENCES

- [1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.
- [2] G. Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger,” Ethereum Project, Yellow Paper 151, 2014.
- [3] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, “Hyperledger fabric: a distributed operating system for permissioned blockchains,” in *Proceedings of the Thirteenth EuroSys Conference*, ser. EuroSys ’18. New York, NY, USA: Association for Computing Machinery, 2018, event-place: Porto, Portugal. [Online]. Available: <https://doi.org/10.1145/3190508.3190538>
- [4] K. Dunnett, S. Pal, G. D. Putra, Z. Jadidi, and R. Jurdak, “A Trusted, Verifiable and Differential Cyber Threat Intelligence Sharing Framework using Blockchain,” in *2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, Dec. 2022, pp. 1107–1114, iSSN: 2324-9013. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10063731>
- [5] P. Huff and Q. Li, “A Distributed Ledger for Non-attributable Cyber Threat Intelligence Exchange,” in *Security and Privacy in Communication Networks*, J. Garcia-Alfaro, S. Li, R. Poovendran, H. Debar, and M. Yung, Eds. Cham: Springer International Publishing, 2021, pp. 164–184.
- [6] Y. Allouche, N. Tapas, F. Longo, A. Shabtai, and Y. Wolfsthal, “TRADE: TRusted Anonymous Data Exchange: Threat Sharing Using Blockchain Technology,” Mar. 2021, arXiv:2103.13158 [cs]. [Online]. Available: <http://arxiv.org/abs/2103.13158>
- [7] S. Gong and C. Lee, “BLOCIS: Blockchain-Based Cyber Threat Intelligence Sharing Framework for Sybil-Resistance,” *Electronics*, vol. 9, no. 3, p. 521, Mar. 2020, number: 3 Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: <https://www.mdpi.com/2079-9292/9/3/521>
- [8] P. S. o. Ethereum, “PSE Roadmap: 2025 and Beyond | PSE.” [Online]. Available: <https://pse.dev/blog/pse-roadmap-2025>
- [9] “A Blockchain Platform for the Enterprise — Hyperledger Fabric Docs main documentation.” [Online]. Available: <https://hyperledger-fabric.readthedocs.io/en/release-2.5/>
- [10] B. Chor, E. Kushilevitz, O. Goldreich, and M. Sudan, “Private information retrieval,” *J. ACM*, vol. 45, no. 6, pp. 965–981, Nov. 1998, place: New York, NY, USA Publisher: Association for Computing Machinery. [Online]. Available: <https://doi.org/10.1145/293347.293350>
- [11] I. Goldberg, “Improving the Robustness of Private Information Retrieval,” in *2007 IEEE Symposium on Security and Privacy (SP ’07)*, May 2007, pp. 131–148, iSSN: 2375-1207. [Online]. Available: <https://ieeexplore.ieee.org/document/4223220>
- [12] A. Beimel, Y. Ishai, E. Kushilevitz, and J.-F. Raymond, “Breaking the $O(n \supset 1/(2k-1))$ barrier for information-theoretic Private Information Retrieval,” in *The 43rd Annual IEEE Symposium on Foundations of Computer Science, 2002. Proceedings.*, 2002, pp. 261–270.
- [13] C. Devet, I. Goldberg, and N. Heninger, “Optimally Robust Private Information Retrieval,” in *21st USENIX Security Symposium (USENIX Security 12)*. Bellevue, WA: USENIX Association, Aug. 2012, pp. 269–283. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/devet>
- [14] D. Demmler, A. Herzberg, and T. Schneider, “RAID-PIR: Practical Multi-Server PIR,” in *Proceedings of the 6th edition of the ACM Workshop on Cloud Computing Security*, ser. CCSW ’14. New York, NY, USA: Association for Computing Machinery, Nov. 2014, pp. 45–56. [Online]. Available: <https://doi.org/10.1145/2664168.2664181>
- [15] S. Angel, H. Chen, K. Laine, and S. Setty, “Pir with compressed queries and amortized query processing,” in *2018 IEEE Symposium on Security and Privacy (SP)*, 2018, pp. 962–979.
- [16] I. Ahmad, Y. Yang, D. Agrawal, A. El Abbadi, and T. Gupta, “Addra: Metadata-private voice communication over fully untrusted infrastructure,” in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, Jul. 2021, pp. 313–329.
- [17] M. H. Mughees, H. Chen, and L. Ren, “Onionpir: Response efficient single-server pir,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 2292–2306. [Online]. Available: <https://doi.org/10.1145/3460120.3485381>
- [18] S. J. Menon and D. J. Wu, “Spiral: Fast, high-rate single-server pir via the composition,” in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 930–947.
- [19] A. Beimel, Y. Ishai, and T. Malkin, “Reducing the servers’ computation in private information retrieval: Pir with preprocessing,” *J. Cryptol.*, vol. 17, no. 2, p. 125–151, Mar. 2004. [Online]. Available: <https://doi.org/10.1007/s00145-004-0134-y>
- [20] A. Henzinger, M. M. Hong, H. Corrigan-Gibbs, S. Meiklejohn, and V. Vaikuntanathan, “One server for the price of two: Simple and fast Single-Server private information retrieval,” in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 3889–3905. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/henzinger>
- [21] M. Zhou, A. Park, W. Zheng, and E. Shi, “Piano: Extremely Simple, Single-Server PIR with Sublinear Server Computation,” in *2024 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2024, pp. 4296–4314. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/SP54263.2024.00055>
- [22] B. Li, D. Micciancio, M. Raykova, and M. Schultz-Wu, “Hintless single-server private information retrieval,” in *Advances in Cryptology – CRYPTO 2024*, L. Reyzin and D. Stebila, Eds. Cham: Springer Nature Switzerland, 2024, pp. 183–217.
- [23] S. J. Menon and D. J. Wu, “Ypir: high-throughput single-server pir with silent preprocessing,” in *Proceedings of the 33rd USENIX Conference on Security Symposium*, ser. SEC ’24. USA: USENIX Association, 2024.
- [24] W.-K. Lin, E. Mook, and D. Wichs, “Doubly efficient private information retrieval and fully homomorphic ram computation from ring lwe,” in *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, ser. STOC 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 595–608. [Online]. Available: <https://doi.org/10.1145/3564246.3585175>
- [25] H. Okada, R. Player, S. Pohmann, and C. Weinert, “Towards practical doubly-efficient private information retrieval,” in *Financial Cryptography and Data Security*, J. Clark and E. Shi, Eds. Cham: Springer Nature Switzerland, 2025, pp. 264–282.
- [26] H. Corrigan-Gibbs and D. Kogan, “Private information retrieval with sublinear online time,” in *Advances in Cryptology – EUROCRYPT 2020*, A. Canteaut and Y. Ishai, Eds. Cham: Springer International Publishing, 2020, pp. 44–75.
- [27] J. Xiao, J. Chang, L. Lin, B. Li, X. Dai, Z. Xiong, K.-K. R. Choo, K. Gai, and H. Jin, “Cloak: Hiding Retrieval Information in Blockchain Systems via Distributed Query Requests,” *IEEE Transactions on Services Computing*, vol. 17, no. 06, pp. 3213–3226, Nov. 2024, place: Los Alamitos, CA, USA

- Publisher: IEEE Computer Society. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/TSC.2024.3411450>
- [28] M. Mazmudar, S. Veitch, and R. A. Mahdavi, "Peer2pir: Private queries for ipfs," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 4438–4456.
 - [29] K. Qin, H. Hadass, A. Gervais, and J. Reardon, "Applying private information retrieval to lightweight bitcoin clients," in *2019 Crypto Valley Conference on Blockchain Technology (CVCBT)*, 2019, pp. 60–72.
 - [30] G. Kumar, R. Saha, M. Gupta, and T. H. Kim, "BRON: A blockchained framework for privacy information retrieval in human resource management," *Heliyon*, vol. 10, no. 13, Jul. 2024, publisher: Elsevier. [Online]. Available: [https://www.cell.com/heliyon/abstract/S2405-8440\(24\)09424-6](https://www.cell.com/heliyon/abstract/S2405-8440(24)09424-6)
 - [31] G. Kumar, R. Saha, M. Conti, and T. H. Kim, "DEBPIR: enhancing information privacy in decentralized business modeling," *Complex & Intelligent Systems*, vol. 11, no. 7, p. 290, May 2025. [Online]. Available: <https://doi.org/10.1007/s40747-025-01868-y>
 - [32] "Lattigo v6," Aug. 2024, published: Online: <https://github.com/tuneinsight/lattigo>. [Online]. Available: <https://github.com/tuneinsight/lattigo>
 - [33] M. Brandenburger, C. Cachin, R. Kapitza, and A. Sorniotti, "Blockchain and Trusted Computing: Problems, Pitfalls, and a Solution for Hyperledger Fabric," 2018, eprint: 1805.08541. [Online]. Available: <https://arxiv.org/abs/1805.08541>
 - [34] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," 2014, eprint: 1407.3561. [Online]. Available: <https://arxiv.org/abs/1407.3561>
 - [35] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *Cryptology ePrint Archive*, Paper 2012/144, 2012. [Online]. Available: <https://eprint.iacr.org/2012/144>
 - [36] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "Fully homomorphic encryption without bootstrapping," *Cryptology ePrint Archive*, Paper 2011/277, 2011. [Online]. Available: <https://eprint.iacr.org/2011/277>
 - [37] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," in *Advances in Cryptology - ASIACRYPT 2017*, T. Takagi and T. Peyrin, Eds. Cham: Springer International Publishing, 2017, pp. 409–437.
 - [38] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "TFHE: Fast fully homomorphic encryption over the torus," *Cryptology ePrint Archive*, Paper 2018/421, 2018. [Online]. Available: <https://eprint.iacr.org/2018/421>
 - [39] C. Peikert, "Lattice Cryptography for the Internet," in *Post-Quantum Cryptography*, M. Mosca, Ed. Cham: Springer International Publishing, 2014, pp. 197–219.
 - [40] C. Mouchet, J. Troncoso-Pastoriza, J.-P. Bossuat, and J.-P. Hubaux, "Multiparty Homomorphic Encryption from Ring-Learning-With-Errors," 2020, published: *Cryptology ePrint Archive*, Paper 2020/304. [Online]. Available: <https://eprint.iacr.org/2020/304>
 - [41] "Ciphertext indistinguishability," Apr. 2025, page Version ID: 1285918512. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Ciphertext_indistinguishability&oldid=1285918512
 - [42] "lattigo/examples at main · tuneinsight/lattigo." [Online]. Available: <https://github.com/tuneinsight/lattigo/tree/main/examples>
 - [43] R. A. Mahdavi, S. Patel, J. Y. Seo, and K. Yeo, "InsPIRe: Communication-Efficient PIR with Server-side Preprocessing," 2025, published: *Cryptology ePrint Archive*, Paper 2025/1352. [Online]. Available: <https://eprint.iacr.org/2025/1352>
 - [44] "client package - github.com/hyperledger/fabric-gateway/pkg/client - Go Packages." [Online]. Available: <https://pkg.go.dev/github.com/hyperledger/fabric-gateway/pkg/client>



ARTUR IASENOVETS is currently pursuing the M.S. degree in the School of Cybersecurity and Information Law at the Chongqing University of Posts and Telecommunications. His research interests include homomorphic encryption, systems for private search, retrieval, and processing of data.



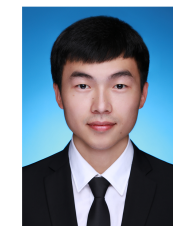
FEI TANG received the Ph.D. degree in information security from the Institute of Information Engineering, Chinese Academy of Sciences, in 2015. He is currently a Professor with the Chongqing University of Posts and Telecommunications. His research interests include public-key cryptography, data security, and so on.



HUIHUI ZHU is currently working toward a Ph.D. degree in the School of Computer Science and Technology, Chongqing University of Posts and Telecommunications, Chongqing, China. His recent research interests include secure multi-party computation and information security.



PING WANG received her M.S. degree from Chongqing University of Posts and Telecommunications in 2022. She is currently pursuing a Ph.D. degree in the School of Computer Science and Technology at Chongqing University of Posts and Telecommunications. Her present research interest includes privacy-preserving computing.



LEI LIU received his M.S. degree in Integrated Circuit Engineering from Fuzhou University in 2018. He is currently pursuing a Ph.D. degree in the School of Computer Science and Technology at Chongqing University of Posts and Telecommunications. His research interests include data security and blockchain.